

// project notes

S3 Static Website Hosting

Complete personal reference — every step, every command, what it does and why.

aws s3 · static hosting · bucket policy · aws cli

// 01 overview

What We Built

A fully public static website hosted directly on Amazon S3 — no server, no EC2, no Nginx. S3 serves HTML/CSS/JS files straight to the browser. The bucket is configured for static website hosting with a custom bucket policy that allows public read access. Files are uploaded using the AWS CLI **aws s3 sync** command.

Local machine (VS Code)	→ git push / aws s3 sync	S3 Bucket (us-east-1)
index.html, style.css, app.js		Static Website Hosting enabled
		Bucket Policy → Public Read
		↓
		http://bucket-name.s3-website-us-east-1.amazonaws.com

// 02 aws console

Part 1 — Creating the S3 Bucket

Step 1 — Open S3 in the AWS Console

Go to Services → S3 → Create bucket.

Why: S3 (Simple Storage Service) is AWS's object storage. When static website hosting is enabled, it acts as a web server — no EC2 needed.

Step 2 — Name the bucket

Choose a globally unique name, e.g. `siddhesh-portfolio-2025`.

Select the region (e.g. `us-east-1`).

Why: S3 bucket names are globally unique across all AWS accounts. The name becomes part of the website URL, so keep it lowercase with hyphens only.

Step 3 — Unblock public access

Under 'Block Public Access settings', uncheck **Block all public access**.

Acknowledge the warning and confirm.

Why: By default AWS blocks all public access to S3 buckets as a security guardrail. Since we want the website to be publicly accessible on the internet, we need to disable this. Only do this for buckets intentionally meant to be public.

→ This alone doesn't make files public — you still need a bucket policy.

Step 4 — Enable Static Website Hosting

After creating the bucket: Properties → Static website hosting → Edit → Enable.

Set Index document: `index.html`

Set Error document: `error.html`

Save changes.

Why: This tells S3 to behave like a web server — serve index.html when someone visits the root URL, and serve error.html for 404/error responses. Without this, S3 just serves raw object URLs, not a proper website.

Step 5 — Note your website endpoint

After enabling, S3 shows your endpoint URL:

```
http://siddhesh-portfolio-2025.s3-website-us-east-1.amazonaws.com
```

This is the public URL of your website. Bookmark it.

// 03 permissions

Part 2 — Bucket Policy (Public Read)

Go to the bucket → Permissions → Bucket Policy → Edit. Paste this policy:

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Sid": "PublicReadGetObject",
      "Effect": "Allow",
      "Principal": "*",
      "Action": "s3:GetObject",
      "Resource": "arn:aws:s3:::siddhesh-portfolio-2025/*"
    }
  ]
}
```

Key	Value	What it means
Version	2012-10-17	Standard IAM policy version – always use this
Effect	Allow	Grant permission (not deny)
Principal	*	Anyone on the internet (all AWS accounts + anonymous)
Action	s3:GetObject	Allow reading/downloading objects (files)
Resource	arn:aws:s3:::bucket/*	Applies to every object inside the bucket

***Why this matters:** IAM policies control access to AWS resources. **Principal: "*" means unauthenticated public access.** **s3:GetObject** only allows reading — nobody can upload, delete, or list your files.*

// 04 aws cli

Part 3 — Uploading Files with AWS CLI

Step 1 — Install & configure AWS CLI

If not installed, download from aws.amazon.com/cli.

Configure with your IAM credentials:

```
aws configure
```

***Why:** The CLI needs your IAM Access Key ID and Secret to authenticate API calls to AWS. Never use root account credentials — create an IAM user with S3 permissions.*

You'll be prompted for:

Prompt	What to enter
AWS Access Key ID	Your IAM user's access key
AWS Secret Access Key	Your IAM user's secret key
Default region name	us-east-1 (or your region)
Default output format	json

Step 2 — Upload all files with aws s3 sync

Navigate to your project folder in terminal, then run:

```
aws s3 sync . s3://siddhesh-portfolio-2025 --delete
```

*Why: **sync** compares local files vs S3 and only uploads what changed — much faster than uploading everything every time. **--delete** removes files from S3 that no longer exist locally, keeping S3 in sync with your local folder.*

Breaking down the command:

Part	Meaning
aws s3 sync	Sync command – compares source and destination
.	Source = current directory (your local project folder)
s3://siddhesh-portfolio-2025	Destination = your S3 bucket
--delete	Delete S3 objects that don't exist locally anymore

Step 3 — Verify the upload

List what's in the bucket:

```
aws s3 ls s3://siddhesh-portfolio-2025
```

→ You should see index.html and other files listed. Then visit your website URL to confirm.

Step 4 — Upload a single file (when you update just one file)

For quick updates without syncing everything:

```
aws s3 cp index.html s3://siddhesh-portfolio-2025/index.html
```

*Why: **cp** copies a single file. Use this for quick one-file fixes. Use **sync** for full deploys.*

// 05 concepts

Key Concepts — How S3 Hosting Works

Concept	Explanation
S3 Object	Any file stored in S3. Has a key (filename/path), data, and metadata.
Bucket	Container for objects. Like a folder, but globally unique and region-specific.
Static hosting	S3 serves files directly as a website – no compute/server involved.
Object URL	Direct link to a file: <code>https://s3.amazonaws.com/bucket/file.html</code>
Website endpoint	The hosted website URL – uses index/error docs, handles routing.

ACL vs Policy	Bucket Policies (JSON) are the modern way. ACLs are legacy – avoid them.
Eventual consistency	Uploads are usually instant but S3 is eventually consistent globally.

// 06 comparison

S3 vs EC2 Hosting — Key Differences

	S3 Static Hosting	EC2 + Nginx
Web server	S3 built-in (managed)	Nginx (you install & configure)
Server access	None – fully managed	Full Linux shell access
Deployment	aws s3 sync	SSH + git pull
Cost (low traffic)	Near-zero (~\$0.023/GB storage)	t2.micro Free Tier, then ~\$8/mo
Scaling	Automatic (AWS handles it)	Manual (you configure Auto Scaling)
HTTPS	Via CloudFront (extra setup)	Via Certbot / ALB (extra setup)
Custom domain	Route 53 + CloudFront	Route 53 → EC2 IP
Dynamic content	Not possible (static only)	Yes – run Node, Python, etc.
Config files	None (just bucket policy)	Nginx config, systemd, etc.
Best for	Portfolios, docs, SPAs	Apps needing server-side logic

// 07 reference

Useful CLI Commands

```
# Sync local folder to S3 (full deploy)
aws s3 sync . s3://your-bucket-name --delete

# Copy single file
aws s3 cp index.html s3://your-bucket-name/index.html

# List bucket contents
aws s3 ls s3://your-bucket-name

# List with file sizes (human readable)
aws s3 ls s3://your-bucket-name --human-readable

# Delete a specific file from S3
aws s3 rm s3://your-bucket-name/old-file.html

# Delete everything in the bucket (careful!)
aws s3 rm s3://your-bucket-name --recursive

# Download entire bucket to local folder
aws s3 sync s3://your-bucket-name ./local-backup

# Check bucket website config
aws s3api get-bucket-website --bucket your-bucket-name

# Check bucket policy
aws s3api get-bucket-policy --bucket your-bucket-name
```

// 08 quick ref

Important URLs & Info

What	Value
Bucket name	siddhesh-portfolio-2025
Region	us-east-1
Website URL	http://siddhesh-portfolio-2025.s3-website-us-east-1.amazonaws.com
S3 Console URL	https://s3.console.aws.amazon.com/s3/buckets/siddhesh-portfolio-2025
Index document	index.html
Error document	error.html

// 09 next steps

Possible Next Steps

→ [Add HTTPS via CloudFront](#)

Put a CloudFront distribution in front of S3. CloudFront provides HTTPS, global CDN caching, and a custom domain. Required if you want https:// instead of http://.

→ **Custom domain with Route 53**

Register or import a domain in Route 53. Create an Alias record pointing to your CloudFront distribution (or S3 website endpoint for HTTP-only).

→ **Automate deploys with GitHub Actions**

Add a workflow: on push to main → run aws s3 sync. Store AWS credentials as GitHub Secrets (AWS_ACCESS_KEY_ID, AWS_SECRET_ACCESS_KEY). Never hardcode credentials in workflow files.

→ **Redo S3 + CloudFront + Route 53 with Terraform**

Provision the entire setup as Infrastructure as Code — bucket, policy, CloudFront distribution, ACM certificate, and Route 53 records. Great Terraform project for the portfolio.

→ **Enable S3 access logging**

Turn on server access logging to track who's hitting your website. Logs go to a separate S3 bucket. Useful for understanding traffic patterns.